

Media Engineering and Technology Faculty
German University in Cairo



LLM Hallucination Detection via Knowledge Augmentation

Bachelor Thesis

Author: Mohammed Mahmoud Elnaggar
Supervisors: Dr. Nourhan Ehab
Submission Date: XX July, 20XX

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree
- (ii) due acknowledgement has been made in the text to all other material used

Mohammed Mahmoud Elnaggar
XX July, 20XX

Acknowledgments

Text

Abstract

Abstact

Contents

Acknowledgments	V
1 Introduction	1
1.1 Section Name	1
1.2 Another Section	1
2 Background and Related Works	3
2.1 Understanding hallucinations in Large Language Models	3
2.1.1 Hallucinations definition in LLM	3
2.1.2 LLM hallucinations types and categories	4
2.1.3 Causes of LLM Hallucinations from Training Data	4
2.1.4 Hallucination detection benchmarks	5
2.2 Knowledge Augmentation Approaches	6
2.2.1 Retrieval-Augmented Generation	6
2.2.1.1 Components	7
2.2.1.2 Models	7
2.2.2 Knowledge Graphs as Structured Knowledge Repositories	7
2.2.2.1 Definition and Representational Framework	7
2.2.2.2 Integration with Large Language Models	8
2.3 Controlling and Constraining LLM Output	8
2.3.1 White-Box Approaches: Leveraging Logit Access	8
2.3.2 Black-Box Control: Neuro-Symbolic Approaches for Constrained Generation	9
2.4 Black-Box LLM Hallucination Detection	10
2.4.1 Zero-Resource Knowledge Hallucination Detection	10
2.4.2 External Knowledge-Based Hallucination Detection	12
3 KGVS: Our method to detect hallucination	15
3.1 System Implementation	15
3.1.1 Hallucination Categories	17
3.1.1.1 Factual Inconsistency Detection	17
3.1.1.2 Logical Inconsistency Detection	18
3.2 Triple Extraction	18
3.3 Triple Predicate Canonicalization	19

3.3.1	Canonicalization Prompt Design	19
3.3.2	Model Selection Experiment	20
3.4	Detect Logical Inconsistency	22
3.4.1	Isolation Test Design	22
3.4.2	Model Evaluation Results	23
3.5	Fetch Evidence	23
3.6	Detect Factual Inconsistency	24
4	Future Work	27
	Appendix	28
A	Lists	29
	List of Abbreviations	29
	List of Figures	30
	References	31

Chapter 1

Introduction

1.1 Section Name

Some sample text with an Acronym Without Citation (AC), some citation ?, and some more Acronym With Citation ? (AC2).

1.2 Another Section

Reference to Section 1.1, and reuse of AC nad AC2 with also full use of Acronym With Citation ? (AC2).

Chapter 2

Background and Related Works

This chapter explains the foundations needed to understand the approach used in this thesis: a knowledge-grounded verification layer that detects hallucinations in LLM outputs by checking them against explicit symbolic knowledge. We commence with Section 2.1, where we define hallucinations in LLMs, discuss their types and causes, and review the benchmarks used to evaluate detection approaches. Subsequently, in Section 2.2, we explore Retrieval-Augmented Generation (RAG) as a knowledge augmentation architecture, which motivates our approach where we will replace retrieval with symbolic constraint checking, and we introduce knowledge graphs as the structured knowledge representation that will serve as the backbone of our proposed approach. Following this, Section 2.3 examines how we can control LLM outputs, particularly in black-box settings where control typically occurs after generation, as it has several connections to black-box hallucination detection. We include white-box approaches in this section primarily to compare the different methodologies and understand how they differ. In Section 2.4, we finally discuss papers that present approaches for detecting hallucinations in black-box LLMs, covering both zero-resource and external knowledge-based methods.

2.1 Understanding hallucinations in Large Language Models

This section covers the foundational concepts of LLM hallucinations, including their definition, types, causes, and the benchmarks used to evaluate detection approaches.

2.1.1 Hallucinations definition in LLM

In the current era, while we are in the evolution in LLMs that generate fluent responses across various tasks, there is a critical problem that may have a significant impact on certain crucial fields where the information provided in the responses of different LLMs

must be factual. Alongside the evolution of LLMs and their fluent responses, LLMs remain prone to hallucinations. LLM hallucination refers to the phenomenon where an LLM produces responses that are unfaithful or nonsensical relative to the source content [Ji et al., 2023, Huang et al., 2025]. Furthermore, hallucinations have become increasingly difficult to detect, as LLMs produce fluent responses, which may lead to harmful consequences.

2.1.2 LLM hallucinations types and categories

First, there are two traditional types of hallucinations: intrinsic and extrinsic [Ji et al., 2023]. Intrinsic hallucination means that the generated output contradicts the source content — for example, when summarizing a text and contradicting a date mentioned in the original content. Extrinsic hallucination means that the generated output cannot be verified against the source content or an external knowledge base. Second, in more recent papers, there is a further categorization of LLM hallucination types into factuality hallucinations and faithfulness hallucinations, each of which is further subdivided [Huang et al., 2025]. For factuality hallucinations, there are two types: the first is factual contradiction, where the LLM generates output that contradicts facts grounded in reality, and the second is factual fabrication, which refers to generated output that cannot be verified against any known source. As for faithfulness hallucinations, they are categorized into three types. The first is instruction inconsistency, where the LLM output deviates from the user’s given directions. The second is context inconsistency, where the LLM output is unfaithful to the contextual information provided by the user. The third is logical inconsistency, where the LLM exhibits internal logical contradictions within its own output.

2.1.3 Causes of LLM Hallucinations from Training Data

In this section, we explore the root cause of hallucinations in LLMs from data [Huang et al., 2025, Ji et al., 2023].

The data used for training LLMs comprises two primary categories: (1) pre-training data, through which LLMs acquire most of their general knowledge and factual information, and (2) alignment data, which teaches LLMs to align with human preferences [Huang et al., 2025].

Misinformation and Biases Fake news and unfounded data have spread widely across the internet, comprising a significant portion of LLM training data and consequently causing hallucinations. Additionally, certain biases related to hallucination have been observed, particularly those associated with gender and nationality [Huang et al., 2025].

Knowledge Boundaries Although LLMs are equipped with extensive factual knowledge, they inherently possess knowledge boundaries. The long-tail boundary arises from the non-uniform distribution of training data, which causes LLMs to demonstrate varying levels of proficiency across different domains. The up-to-date problem occurs because once an LLM is trained, its knowledge becomes outdated, and factual information that changes over time may lead to hallucinations [Huang et al., 2025].

Copyright-Sensitive Knowledge Since LLMs rely on publicly available data, limitations arise in domains that contain inaccessible or restricted data, which causes hallucinations in those specific domains [Huang et al., 2025].

Inferior Alignment LLMs struggle to acquire new knowledge during Supervised Fine-Tuning (SFT), and a correlation has been identified between the knowledge acquired during SFT and the occurrence of hallucinations [Huang et al., 2025].

2.1.4 Hallucination detection benchmarks

Previous benchmarks were focused on models with specific tasks and were not general purpose. The benchmarks were evaluating smaller and task-specific models which produce errors which are different from new era LLMs, since benchmarks used were on smaller task-specific models which is not compatible with today’s LLMs and cannot be used to evaluate how well detection methods used on today’s powerful LLMs; these benchmarks must be changed [Huang et al., 2025].

A set of benchmarks has since been introduced to evaluate how well hallucination detection methods work, in this part we discuss some of the modern datasets to evaluate hallucination detection approaches.

HaluEval: A large-scale benchmark introduced by [Li et al., 2023] which is a large collection of 35,000 generated and human-annotated samples to evaluate LLMs’ ability to recognize hallucinations. The benchmark covers 30,000 samples across multiple tasks including question answering, knowledge-grounded dialogue, and text summarization, providing a comprehensive evaluation framework for hallucination detection across different LLM applications, along with 5,000 samples of general user queries with ChatGPT responses.

FELM: A benchmark introduced by [Chen et al., 2023] for evaluating factuality evaluation methods themselves, rather than directly evaluating LLMs. It contains 2,000+ responses from various LLMs across different domains, each annotated with fine-grained factuality labels, enabling systematic assessment of how well different evaluation metrics and detectors perform at identifying factual errors.

SelfCheckGPT-Wikibio: The WikiBio GPT-3 dataset introduced by [Manakul et al., 2023] offers a sentence-level dataset created by GPT-3 on individuals from the WikiBio dataset, with manual annotation of factuality. WikiBio is a dataset where each input is the first paragraph of Wikipedia articles of specific concepts. They sampled 238 of the top 20 percent of longest articles.

BAMBOO: A benchmark proposed by [Dong et al., 2023] which is a multi-task long-context benchmark used to evaluate long-text tasks including hallucination detection. BAMBOO consists of 10 different datasets from 5 different long-text understanding tasks: question answering, hallucination detection, text sorting, language modeling, and code completion.

ScreenEval: A benchmark introduced by [Lattimer et al., 2023] for evaluating factual inconsistencies in long dialogues. This dataset comprises 52 dialogues averaging 6,000 tokens per dialogue. The dataset uses TV scripts and summaries pulled from the SummScreen dataset; they also generate summaries using Longformer and GPT-4 on 52 scripts from the SummScreen test set.

PHD: A passage-level hallucination detection benchmark introduced by [Yang et al., 2023] containing 300 samples generated by ChatGPT on Wikipedia articles, with human annotations at both sentence and passage levels. The paper also introduces a reverse validation method that reconstructs questions from generated passages and verifies consistency, specifically designed to address the omission problem in traditional consistency-based detection.

LSum: A benchmark introduced by [Feng et al., 2023] for evaluating hallucination in text summarization tasks. The paper proposes an adversarial approach that decouples comprehension and embellishment abilities of LLMs, demonstrating that factual consistency can be improved by separating what the model understands from what it generates, with applications for both detection and mitigation of hallucinations in summarization.

2.2 Knowledge Augmentation Approaches

2.2.1 Retrieval-Augmented Generation

Pre-trained language models have been shown to learn a substantial amount of in-depth knowledge from data, and they are capable of doing so without accessing any external memory. However, such models cannot expand or revise their knowledge after training, which may lead to hallucinations [Lewis et al., 2020]. This thesis draws on the same motivation as RAG — grounding LLM outputs in external knowledge — but replaces retrieval with symbolic constraint checking against a structured knowledge representation.

2.2.1.1 Components

RAG consists of two core components: the Retriever and the Generator [Lewis et al., 2020].

- **Retriever:** Responsible for returning the top- k documents based on a truncated distribution over the input sequence X .
- **Generator:** Utilizes the retrieved documents as additional context to generate the target sequence Y .

2.2.1.2 Models

- **RAG Sequence Model:** The RAG sequence model uses the same set of retrieved documents to generate the entire target sequence [Lewis et al., 2020].
- **RAG Token Model:** The RAG token model uses different retrieved documents for each individual target token, allowing for more fine-grained document selection during generation [Lewis et al., 2020].

2.2.2 Knowledge Graphs as Structured Knowledge Repositories

2.2.2.1 Definition and Representational Framework

Knowledge graphs represent information as entities (nodes) and relations (edges), forming a structured repository of facts. Facts are stored as triples of the form $\langle \textit{subject}, \textit{predicate}, \textit{object} \rangle$, such as $\langle \textit{Paris}, \textit{capitalOf}, \textit{France} \rangle$, which allows structured reasoning over the stored knowledge.

To illustrate, consider the sentence "Paris is the capital of France." This would be stored as the triple (Paris, capitalOf, France). If an LLM generates the response "Paris is the capital of Germany," a verification system can query the knowledge graph for (Paris, capitalOf, ?). The graph returns "France," which contradicts the generated "Germany," signaling a hallucination. This query-based mechanism is how knowledge graphs enable fact-level verification.

In the context of hallucination detection, knowledge graphs are used as a way of storing rules or facts that will be verified or assigned a score [Sawczyn et al., 2025, Kale and Alfeo, 2025, Fang et al., 2024].

2.2.2.2 Integration with Large Language Models

Recent works integrate knowledge graphs with LLMs to improve factual accuracy and detect hallucinations. The integration typically follows two patterns. The first pattern constructs graphs from LLM responses and uses them for structural consistency checking. For instance, [Fang et al., 2024] constructs graphs from responses and uses graph neural networks to model contextual dependencies between triples, enabling detection of logical inconsistencies across multiple facts. The second pattern uses knowledge graphs as an external reference to verify extracted facts. [Sawczyn et al., 2025] and [Kale and Alfeo, 2025] leverage knowledge graphs for fine-grained fact-level consistency checking by comparing triples extracted from LLM outputs against triples stored in the graph.

2.3 Controlling and Constraining LLM Output

Enforcing domain rules and logical constraints on LLM output is a major concern in modern research, as it makes LLMs more reliable. While this thesis focuses on detecting hallucinations after generation, examining methods that prevent hallucinations through controlled generation provides valuable context. Both control and detection share a common goal: ensuring LLM outputs align with expected facts or rules. Additionally, some detection approaches borrow techniques from constrained decoding, particularly in how they interact with external knowledge sources. There are several approaches for controlling LLM output: ones that are used with white-box LLMs and ones that are used with black-box LLMs, where access to their logits is unavailable. In this section we discuss each one of them, with emphasis on black-box methods as they are most relevant to the detection approaches surveyed in Section 2.4.

2.3.1 White-Box Approaches: Leveraging Logit Access

The techniques used to control LLM output share the same goal: enforcing $p(x|a)$, where x is the output sequence and a is the constraint that needs to be satisfied.

There are papers that distill a TPM (Tractable Probabilistic Model), most commonly an HMM (Hidden Markov Model), then combine the HMM with the LLM’s output probability distribution and search ahead to determine whether the given constraint will be enforced. One such paper, GeLaTo, trains an HMM via maximum likelihood estimation on samples drawn from LLM output, and at generation time combines the HMM probability with the LLM output [Zhang et al., 2023a]. Another paper, Ctrl-G, generalizes GeLaTo by distilling an HMM, constructing a DFA, and then conditioning the HMM on the DFA [Lu et al., 2023]. Yet another paper distills an HMM and combines it with an efficiently trained classifier that guides the computation of the Expected Attribute Probability, which in turn steers generation [Wang et al., 2022].

Another approach is presented in the paper FUDGE, which shares the same goal of modeling $p(x|a)$, but instead of using a TPM, it trains a binary predictor on whether

constraint a will be satisfied based on an incomplete sequence prefix [Yang and Klein, 2021]. It multiplies the probability distributions of the predictor and the LLM output, then selects the token with the highest weight.

These white-box methods share a common pattern: they all require access to the LLM’s internal logits and involve training auxiliary models (HMMs or binary predictors) to guide generation. While they are effective, their reliance on logit access limits their work with black-box LLMs.

2.3.2 Black-Box Control: Neuro-Symbolic Approaches for Constrained Generation

In the previous section, constraints were applied by leveraging access to the LLM’s probability distribution. However, when using black-box LLMs such as ChatGPT or Claude, access to the model’s internal logits is unavailable, so alternative approaches are needed to enforce rules or logical constraints on the output of such black-box LLMs.

One such technique is presented in the paper SketchGCD, which proposes an approach for constrained decoding on LLM output that requires no further training [Lu et al., 2024]. It uses a lightweight open-source model to refine the output of a heavyweight black-box LLM to satisfy specific constraints. This approach is divided into two components: a sketcher and a constrained decoder. For the constraints to be applied, it restricts entities and relations to the Wikidata knowledge graph and enforces a structural format that the output must follow.

Another technique that integrates LLMs with symbolic solvers to improve logical problem-solving is LOGIC-LM [Pan et al., 2023], which takes a problem P and a goal G and feeds them into a multi-stage process to derive the answer. First, it prompts an LLM to translate P and G into task-specific symbolic representations, using four different symbolic formulations: logic programming, first-order logic, constraint satisfaction problems, and analytical reasoning. After converting P and G into symbolic representations, they are passed to a symbolic solver to obtain answer A by enforcing logical rules. The approach also includes a self-refinement module: if the solver returns a logical or syntactic error, the LLM revises the formulation and repeats the process iteratively.

A third approach used to control the generated content at the token level of black-box LLMs introduces ScoPE (Score-based Progressive Editor), proposed by [Yu et al., 2024]. This method employs an editor module that edits blocks of size b tokens from the text generated by a black-box LLM, evaluating how well a target attribute is incorporated into the text. ScoPE follows an iterative process of progressively editing the output text to align it with the target domain, using score functions based on fine-tuned masked language models and task-specific discriminators. The editor operates without access to model parameters or logits, making it compatible with commercial LLM APIs, and is trained to maximize the disparity in target domain scores between input and edited text while preserving fluency through techniques such as teacher forcing and selective token-position training [Yu et al., 2024].

A fourth approach is presented in the paper GraphCorrect [Sansford et al., 2024]. First, triples extracted from the LLM output are detected for hallucination using GraphEval, which is discussed in the same paper and will be addressed later in this section 2.4. GraphCorrect proceeds in two stages. In the first stage, hallucinated triples are provided to the LLM along with the structured context, prompting it to correct the hallucination and output a newly generated triple. In the second stage, the original triple and its corrected counterpart are taken together with the initial LLM output, and the information from the original triple is replaced with the corrected version in the initial LLM output.

These black-box control methods demonstrate that logical and factual constraints can be enforced on LLM outputs without internal logit access. Notably, approaches like Logic-LM integrate symbolic solvers for logical verification, while SketchGCD relies on knowledge graphs (Wikidata) as the source of constraints—the same type of structured knowledge that this thesis uses for detection. Additionally, ScoPE shows that score-based editing with domain-specific MLMs offers another viable path for enforcing constraints, such as topic or sentiment control, without external knowledge resources. GraphCorrect adds a further dimension by demonstrating a post-hoc correction mechanism that first detects hallucinations and then repairs them using structured context, effectively bridging the gap between detection and correction. The key difference across these methods is one of timing and mechanism: control methods intervene during generation to prevent hallucinations (whether through sketching, symbolic solving, or progressive editing), while methods like GraphCorrect operate after generation to identify and fix them. This distinction aside, the underlying goal of ensuring LLM outputs align with expected facts or rules is common to all, making constrained decoding a relevant adjacent field to the hallucination detection approaches discussed next.

2.4 Black-Box LLM Hallucination Detection

As mentioned previously, hallucinations are a major concern in the LLM field, so detecting them is a crucial area of research in which several approaches have been proposed, each employing different techniques. The hallucination detection approaches for black-box LLMs are divided into two parts: zero-resource and external knowledge hallucination detection.

2.4.1 Zero-Resource Knowledge Hallucination Detection

This section covers four different types of zero-resource approaches: 1) SelfCheck zero-resource hallucination detection, 2) fine-grained cross-model evaluation, 3) graph-based context-aware hallucination detection and 4) knowledge graphs for hallucination self-detection. These methods share a common strength: they require no external knowledge base, making them broadly applicable across domains. However, they also share a fundamental limitation that motivates the external knowledge-based approaches discussed in Section 2.4.2.

SelfCheckGPT: One approach is presented in the paper SelfCheckGPT, which requires no external knowledge and uses the LLM to self-check its output, operating effectively with black-box LLMs [Manakul et al., 2023]. This is achieved by obtaining a response R from the LLM given a prompt, then sampling additional responses, which are subsequently used with different methods (BERTScore, NLI, n-grams, etc.) to measure the consistency between the original response and the samples, and then assigning a hallucination score.

FINCH-ZK: Another approach is presented in the paper FINCH-ZK (FINE-grained Cross-model consistency for Hallucination detection and mitigation with Zero Knowledge) [Karatopak et al., 2024]. It requires no external knowledge base to detect hallucinations and uses an LLM to judge the outputs. First, a cross-model sample generation step is performed, in which variants of the prompt are generated and a set of sampler LLMs (M) are used to produce samples. Once these samples are collected, a judge LLM performs fine-grained evaluation. The response is segmented into blocks, and then a judge LLM (J) is used to evaluate each block against each sample, after which hallucination scores are computed.

Graph-based Context-Aware (GCA): The third approach is Graph-based Context-Aware (GCA) zero-resource hallucination detection, which focuses on long text generation with black-box models [Fang et al., 2024]. The paper begins by extracting knowledge triples from the response by providing the LLM with the response and prompting it to extract the triples. Next, it asks the LLM to verify each triple against the original response, and if there is ambiguity in equivalence, it asks the LLM to adjust. Then they construct a graph for the original response and each sampled response, using a Relational Graph Convolutional Network (RGCN) which passes information through nodes, so that when the model checks a triple, it remembers information from other triples and can detect logical errors. After that, they perform graph-based consistency comparison where they compare triples from the original response with triples from sampled responses. If alignment is found, they add 1 to the consistency score, and if there is contradiction between triples, they subtract 1 from the consistency score. Finally, they perform reverse verification through triple reconstruction, where they check if the LLM can correctly reconstruct the head entity, relation, and tail entity of each triple using three different tasks, and combine these scores with the consistency score to make the final hallucination judgment.

Knowledge Graph Self-Detection: The fourth approach involves using knowledge graphs for self detection of hallucinations. One method is presented in the paper Fact-SelfCheck, which operates at the fact level rather than the sentence level [Sawczyn et al., 2025]. It follows a similar procedure to SelfCheckGPT by first generating a response and multiple samples. However, instead of comparing sentences, it constructs knowledge graphs by extracting entities and relations from the response using LLM prompts, does the same with the samples, and then checks the consistency of the extracted facts to assign hallucination scores. Similarly, another approach is presented in the paper Lie to Me, which enhances both single-sample and multi-sample self-detection techniques using knowledge graphs [Kale and Alfeo, 2025]. After obtaining a response, it prompts

the LLM to construct a knowledge graph, and then uses either self-questioning or self-confidence approaches for single-sample detection, or integrates knowledge graphs into SelfCheckGPT for multi-sample detection, to assign hallucination scores at the fact level.

While these zero-resource approaches offer practical advantages, they share a critical limitation: they cannot verify claims that fall outside the LLM’s parametric knowledge. If a model confidently generates a false fact and also repeats it in sampled responses, consistency-based methods may incorrectly label it as factual. This limitation necessitates approaches that consult external knowledge sources, which are discussed next.

2.4.2 External Knowledge-Based Hallucination Detection

While zero-resource approaches offer practical advantages, they cannot verify claims that fall outside their parametric knowledge, which is where external knowledge-based methods are needed.

FactScore: One approach is proposed in the paper FActScore, which evaluates generation based on atomic facts [Min et al., 2023]. This approach uses both human annotation and automated annotation to evaluate atomic facts. For the human annotation, they sampled 183 entities from Wikipedia, fed prompts to LLMs, and then human annotators broke the generation into atomic facts and labeled them as supported, not-supported, or irrelevant based on the knowledge source of Wikipedia. However, this approach is expensive, so they introduced automated annotation where an LLM subject generates text and an LLM evaluator assigns fact scores. The estimator breaks the generation into atomic facts and assigns scores against the knowledge source.

REFCHECKER: Another approach is REFCHECKER, which uses claim-triplets instead of sentence-level evaluation, as one sentence may contain several facts [Hu et al., 2024]. This approach consists of two parts: the extractor and the checker. The extractor extracts triples from the response using models like GPT-4, Mixtral, or a fine-tuned Mistral model trained on 10k responses with Mixtral as a teacher. The checker then compares these triples against a reference to determine if they are entailed, contradicted, or neutral.

GraphEval: There is an approach that uses a knowledge graph and the external knowledge from a given context to match triples [Sansford et al., 2024]. GraphEval focuses on closed-domain hallucination detection, where it first constructs a knowledge graph from the given LLM output, then iterates through the triples and identifies whether they are factually consistent with the given context or not. It uses an NLI model in the second stage of checking triples against the context.

FacTool: A fourth method for detecting factual errors in LLM outputs is introduced in this paper [Chern et al., 2023]. It works through five stages, starting with claim extraction via an LLM. From there, FacTool generates queries for each claim to pull in evidence from external tools — search engines, code interpreters, calculators, or academic search — depending on the task at hand. Each claim is then given a binary factuality label

based on how relevant the gathered evidence is, with the LLM also offering reasoning, flagging errors, and suggesting corrections for any hallucinated claims.

FacTool brings an extra layer to external knowledge-based hallucination detection by combining web search with task-specific tools, though it still depends on verifying against unstructured text. The approaches discussed above all point to the same takeaway: consulting external knowledge is genuinely useful for detecting hallucinations. That said, two of these methods share a notable limitation — they check claims against unstructured text (Wikipedia), which requires natural language understanding to make the comparison work. GraphEval, despite its use of knowledge graphs, runs into a similar issue: it relies on an NLI model to check triples against unstructured contextual text rather than a structured symbolic knowledge base, meaning the ambiguity of natural language interpretation is still present. This thesis takes a different path by using a structured knowledge graph as the reference point, which makes direct triple-to-triple matching possible and sidesteps the interpretive uncertainty that comes with natural language. This distinction is at the heart of the proposed approach laid out in the next chapter.

Chapter 3

KGVS: Our method to detect hallucination

This chapter introduces the design and implementation for the system KGVS (Knowledge Graph Verification System), a neurosymbolic hallucination detection system that detects hallucinations for specific domains. This system combines the capabilities of LLMs to reason and domain-specific knowledge to detect hallucinations produced by LLMs in these specific domains. In this system we use a knowledge graph to store the domain rules and constraints of the domain. This method focuses on detecting factually and logically inconsistent output produced by LLMs.

3.1 System Implementation

The domain rules and constraints are first stored in a graph database (Neo4J), which consists of nodes and relationships and serves as the knowledge base against which hallucinations are assessed. The domain captured in this graph is a consumer credit scoring domain, covering rules such as credit score to interest rate mappings, risk tier assignments, approval odds, and related eligibility constraints — a domain well-suited for symbolic verification due to its clearly formalizable and unambiguous rules.

This system is implemented using LangChain, which provides a straightforward pipeline abstraction that is sufficient for this task. LangGraph was considered as an alternative, but since the pipeline does not require cycles or conditional branching between stages, the additional complexity it introduces was unnecessary. The system uses a structured state dictionary to carry the information produced at each stage forward through the pipeline.

Given a user query containing domain context and a question, the system is designed to address the core research question of this thesis: how can symbolic domain knowledge be used to detect logically or factually inconsistent LLM outputs? To answer this, the query is processed through five main stages. First, the user input is given to an LLM

to produce an answer asking for structured output; a second LLM then extracts the triples from the answer, ensuring that any additional information, whether structured or unstructured, is captured as the necessary triples. Second, the extracted triples are passed to the canonicalization stage, where an LLM try to match each triple’s predicate to the closest predicate in the graph database schema, either canonicalizing it via semantic similarity or marking it as unmatched. Third, the canonicalized triples are examined for internal logical inconsistencies. Fourth, the canonicalized triples are used to retrieve related triples from the database and to fetch the domain constraints, which represent the general rules of the domain. Fifth, the detection of factually inconsistent triples proceeds in two passes: the first pass compares the triples against domain constraints, and the second pass verifies them against the related triples retrieved earlier. The system architecture is depicted in Figure 3.1.

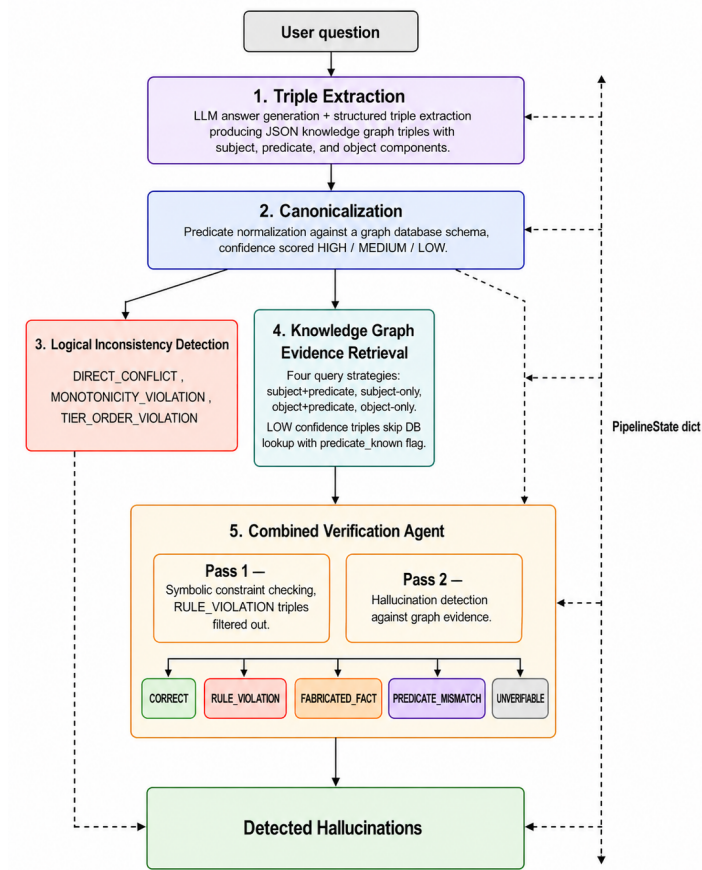


Figure 3.1: Overview of the hallucination detection pipeline. The system processes the user query through five stages: triple extraction, canonicalization, logical inconsistency detection, evidence retrieval, and combined verification (symbolic constraint check followed by factual detection).

3.1.1 Hallucination Categories

Based on the symbolic verification approach, each extracted and canonicalized triple (s, p, o) is assigned to one of two high-level inconsistency types: *factually inconsistent* or *logically inconsistent*. The classification follows a decision flow that first verifies logical inconsistency between multiple triples and then separately verifies factual correctness against the domain constraints and the graph database,

3.1.1.1 Factual Inconsistency Detection

For factual inconsistency, each triple is first checked against the domain constraints stored in Neo4j (e.g., allowable value ranges, type restrictions). If the triple violates a constraint, it is immediately labeled as a **Rule Violation**. Only triples that satisfy all relevant constraints proceed to the next step, where they are compared against the actual factual knowledge in the graph database. Depending on the match pattern, one of the following four categories is assigned.

- A. Unverifiable** The predicate p is not present in the database schema and cannot be matched to any known predicate via semantic similarity during canonicalization. The system lacks the symbolic means to confirm or refute such a triple.
- B. Rule Violation** The subject s and predicate p are supported by the database, but the object o falls outside the explicit value domain or constraint defined for that predicate (e.g., an interest rate of 30% for credit score 300). This category is also used for triples that violate a non-instantiated domain rule (e.g., a credit score of 300 having an approval odds value that is defined only for scores ≥ 400).
- C. Predicate Mismatch** The subject s and predicate p are both present in the database schema, and o is a legal value according to domain constraints, but no triple (s, p, o) exists in the graph database. In other words, this known predicate does not exist for this subject in database.
- D. Fabricated Fact** Two subcases are distinguished:
 - The subject s is not a known entity in the database, while the predicate p and object o are both supported.
 - The object o and subject s is not a known entity in the database, while the and predicate p are supported.

In either subcase, the triple invents an entity that does not exist in the domain knowledge base.

3.1.1.2 Logical Inconsistency Detection

Logical inconsistencies are identified without querying the graph database; they arise from contradictory relations among the triples generated from a single LLM answer. These checks are performed after canonicalization but before factual verification. The following three categories are defined.

- A. Direct Conflict** Two triples share the same subject s and the same predicate p but assign two different, mutually exclusive objects o_1 and o_2 . *Example:* (600, HAS_INTEREST_RATE, 8%) and (600, HAS_INTEREST_RATE, 15%).
- B. Monotonicity Violation** For a predicate that must strictly improve as the subject’s credit score increases (e.g., interest rate should decrease or remain constant), a higher score receives a worse value than a lower score. *Example:* A credit score of 700 is assigned an interest rate of 9.5%, while a score of 580 is assigned 7.2%.
- C. Tier Order Violation** A violation of the expected ordering of risk tiers: a lower credit score receives a better (lower-risk) tier than a higher credit score. *Example:* Score 500 is labelled “super prime” (best tier), whereas score 750 is labelled “subprime” (worst tier).

Triples that pass both the factual and logical consistency checks are considered *consistent* with respect to the symbolic domain knowledge. All other triples are flagged as hallucinations of the corresponding category, providing fine-grained feedback on the nature of the LLM’s error.

3.2 Triple Extraction

In this stage, a primary LLM (Mistral) first takes the user query and produces a structured answer. Mistral was chosen as the fixed answer generator to ensure a consistent and reproducible baseline across all extraction model comparisons; fixing the answer source guarantees that any differences observed in Table 3.1 reflect the extraction model’s capability rather than variation in the generated answer itself. However, because the answering model may embed extra explanations or unstructured text alongside the requested structured output, a second, dedicated extraction LLM is employed. Its task is to guarantee that the final output consists solely of the necessary triples. The stage is deliberately split into answering and extraction because we observed that asking a single LLM to handle both tasks simultaneously often causes it to hallucinate and produce incorrect outputs. Thus, the pipeline proceeds as follows: the first LLM generates an answer, and the second LLM extracts all pertinent triples from that answer.

The extraction itself follows the Extract-Define-Canonicalize (EDC) framework proposed by [Zhang and Soh, 2024]: the LLM first identifies unique entities mentioned in the

answer, then finds the predicates that link those entities, and finally produces a natural-language definition for each predicate. To select the most reliable extraction model, we tested five candidates — GLM-4.7, LLAMA3.3:70B, GPT-4.1, Gemini-2.5-Flash, and Mistral — while keeping Mistral as the fixed answer generator. Five representative questions were used, each targeting a specific part of the domain context — such as interest rate mappings, risk tier assignments, and approval odds — so that together they cover most of the domain context. The expected triples for each question were defined manually before running any model, ensuring that the evaluation was not influenced by model outputs. Table 3.1 summarises the results. Each cell indicates how many of the expected triples were correctly extracted, and whether any additional correct triples or incorrect extractions were produced.

After comparing the outputs, GLM-4.7 was chosen as the extraction model for the final pipeline for several reasons. First, it consistently returned the exact set of required triples and never fabricated a wrong triple across all five questions. Second, on Q2 it contributed extra correct triples beyond the expected set: some were extracted from the LLM’s explanatory description that accompanied the structured answer, and others arose from triples whose object field contained multiple values that GLM-4.7 correctly decomposed into separate triples. While such additional extractions could in principle introduce noise, this risk is controlled downstream by the canonicalization stage, which filters out any triple whose predicate cannot be matched to the graph schema. This behaviour is therefore beneficial for thorough hallucination detection rather than problematic. Third, although GPT-4.1 also produced zero wrong triples, GLM-4.7 was preferred because it is open-source, significantly cheaper to operate at scale, and demonstrated the additional extraction capability on Q2 that GPT-4.1 did not. These combined factors make GLM-4.7 the most suitable choice for the extraction stage.

3.3 Triple Predicate Canonicalization

Continuing the EDC pipeline [Zhang and Soh, 2024], this stage takes the extracted triples and matches their predicates to the formal relationship types defined in the database schema. All valid relationship types are first fetched from the database at runtime. For each triple, its predicate is submitted to an LLM, which identifies the most semantically aligned relationship type from the schema. Since not every extracted predicate has a corresponding schema entry, the model also returns a match confidence level—*High*, *Medium*, or *Low*—where *Low* explicitly signals that no honest match could be found and the triple should be flagged for downstream hallucination detection rather than discarded.

3.3.1 Canonicalization Prompt Design

The canonicalization prompt instructs the model to follow a structured three-step matching procedure. First, qualifying adjectives that narrow meaning without a dedicated

Table 3.1: Extraction model comparison on five pilot questions. “Correct” indicates the number of required triples that were extracted exactly; additional notes describe helpful extras or erroneous outputs.

Question	Expected triples	GPT-4.1	Mistral	LLAMA3.3:70B	GLM-4.7	Gemini-2.5-Flash
Q1	9	9 correct	9 correct	9 correct	9 correct	9 correct
Q2	7	7 correct	+1 add. correct +7 wrong	7 correct	+1 add. correct +2 correct from explanation	+1 add. correct +6 wrong
Q3	15	15 correct	15 correct +4 wrong	15 correct	15 correct	15 correct
Q4	21 (incl. 1 duplicate)	21 correct (with duplicate)	20 correct (without duplicate)	21 correct (with duplicate)	20 correct (without duplicate)	20 correct (without duplicate)
Q5	18	18 correct	18 correct	18 correct	18 correct	18 correct

schema counterpart—such as certainty markers (*estimated*, *confirmed*), temporality markers (*current*, *previous*), and ordinality markers (*minimum*, *maximum*)—are stripped from the extracted predicate to expose its core verb phrase. Second, the core predicate is matched against the schema using exact match, synonym inference, or partial overlap, in that priority order. Third, if a qualifier was stripped to achieve a match, its semantic content is preserved by appending it as a parenthetical annotation to the object field, ensuring no information is silently lost. A strict rejection rule prevents forcing a match: if the core verb phrase shares no overlap with any schema relationship type after all steps, the predicate is returned unchanged in `ALL_CAPS_WITH_UNDERSCORES` form with a *Low* confidence assignment.

3.3.2 Model Selection Experiment

To identify the most reliable canonicalization model, five candidates were evaluated—**GPT-4.1**, **Mistral**, **LLaMA 3.3 70B**, **GLM-4.7**, and **Gemini-2.5-Flash**—while keeping Mistral as the fixed answer generator and GLM-4.7 as the fixed extraction model. The evaluation used a controlled set of 30 synthetic triples, stratified across four difficulty categories designed to probe distinct failure modes:

- **Category A — Direct synonyms** (10 triples): predicates that paraphrase a schema relationship closely enough that an exact or near-exact match is expected.
- **Category B — Partial or ambiguous** (5 triples): predicates carrying qualifying adjectives or indirect phrasing that require qualifier stripping or synonym inference to match.

- **Category C — Clear fabrications** (8 triples): predicates describing concepts entirely absent from the schema, such as historical provenance or demographic associations, which should receive *Low* confidence.
- **Category D — Near-schema traps** (7 triples): predicates that superficially resemble schema relationships—sharing surface-level financial vocabulary—but encode distinct concepts not represented in the database, designed to elicit false-positive matches.

Categories A and B form the 15 matchable triples, while Categories C and D form the 15 that should receive *Low* confidence. Each model was evaluated on two metrics: *match decision accuracy*, measuring whether the model correctly matched or rejected each predicate, and *predicate accuracy*, measuring whether matched predicates were mapped to the correct schema relationship type.

Table 3.2: Canonicalization model evaluation results across 30 test triples.

Model	Match Decision	Predicate Accuracy
GPT-4.1	30/30 (100%)	15/15 (100%)
Mistral	30/30 (100%)	15/15 (100%)
LLaMA 3.3 70B	30/30 (100%)	15/15 (100%)
GLM-4.7	30/30 (100%)	15/15 (100%)
Gemini-2.5-Flash	30/30 (100%)	15/15 (100%)

As shown in Table 3.2, all five models achieved perfect scores on both metrics. Two factors likely explain this outcome. First, the relationship types stored in the database are self-describing: rather than short identifiers such as `HAS_RATE` or `HAS_TIER`, each relationship type encodes its full semantic meaning directly in its name, for example `INTEREST_RATE_CHARGED_AS_ANNUAL_BORROWING_COST_PERCENTAGE` or like the predicate `RISK_TIER_CLASSIFIED_AS_BORROWER_DANGER_LEVEL`. This makes the matching task substantially easier, because even near-synonym or partially overlapping predicates can be resolved by reading the full relationship name, reducing the ambiguity that would arise with abbreviated schema identifiers. Second, the extraction stage produces a natural-language predicate definition alongside each triple, which provides the canonicalization model with additional semantic context beyond the raw predicate string itself. Together, these two factors likely made the task well-constrained enough that model architecture had no measurable effect on accuracy within this candidate set. Since model choice does not affect correctness at this stage, the canonicalization model can be selected based on cost and latency considerations rather than accuracy, meaning smaller and cheaper models are entirely sufficient for this task.

3.4 Detect Logical Inconsistency

In this stage, canonicalized triples are taken and then organized into groups to find logical inconsistencies between triples. There are mainly three types of logical inconsistency that the system is designed to detect. First, **Direct Conflict** occurs when two triples share the same subject and predicate but carry semantically incompatible objects—for example, a credit score of 600 being simultaneously classified as both *prime* and *deep subprime*. Second, **Monotonicity Violation** occurs when a predicate that must respect a monotonic relationship with credit score is violated across different subjects; for instance, interest rate must decrease as the credit score increases, so a higher-scored borrower carrying a higher rate than a lower-scored one constitutes a violation. Third, **Tier Order Violation** occurs when the categorical risk tier assigned to a lower credit score is strictly higher in the tier hierarchy than the one assigned to a higher credit score, which contradicts the fundamental ordering of creditworthiness tiers (*deep subprime* < *subprime* < *near prime* < *prime* < *super prime*).

The detection pipeline first groups the canonicalized triples into three structured sets before passing them to the LLM. The first group, `same_subject_same_predicate`, collects triples that share both the same subject and the same predicate but carry different object values, making them candidates for Direct Conflict. The second group, `same_predicate_different_scores`, gathers triples that share a predicate across different credit score subjects and annotates each predicate with its required direction (either DECREASING, INCREASING, or TIER), providing the LLM with all the information it needs to reason about monotonicity and tier ordering without relying on any external knowledge. The third group, `all_triples`, provides the complete flat list for reference. The LLM receives all three groups as a single structured JSON payload and is instructed to return only genuine contradictions that exist within the triples themselves, with no appeal to external domain facts.

3.4.1 Isolation Test Design

To evaluate the detection capability of each model in isolation, a controlled test set of 21 canonicalized triples was constructed. The test set was carefully designed to contain exactly 2 Direct Conflicts, 3 Monotonicity Violations, and 2 Tier Order Violations, alongside a control group of consistent triples that should produce no false positives. The injected inconsistencies are described below.

The two **Direct Conflicts** are: (1) credit score 600 simultaneously classified as both *prime* and *deep subprime* under the risk tier predicate, and (2) credit score 700 assigned two different annual interest rates of 8.31% and 15% under the same predicate.

The three **Monotonicity Violations** are: (1) credit score 750 carrying a higher interest rate (14.5%) than the lower credit score 580 (11.91%) or 660 (9.49%), violating the decreasing direction requirement; (2) credit score 800 having lower approval odds (30%) than the lower credit score 740 (89%) or 620 (55%), violating the increasing direction

requirement; and (3) credit score 740 carrying a higher default probability (35%) than the lower credit score 400 (28.7%), violating the decreasing direction requirement.

The two **Tier Order Violations** are: (1) credit score 350 classified as *super prime* while credit score 800 is classified as *subprime*, and (2) credit score 500 classified as *prime* while the higher credit score 580 is classified as *subprime*.

The control group includes triples with equal values across different scores (which must not be flagged) and correctly ordered triples that serve as negative examples.

3.4.2 Model Evaluation Results

Five large language models were evaluated on the isolation test: Gemini, GLM, GPT-4o, LLaMA, and Mistral. Each model received the same grouped triple payload and was expected to return exactly 2 Direct Conflicts, 3 Monotonicity Violations, 2 Tier Order Violations, and 0 false positives. Table 3.3 summarizes the results for each model across all four evaluation criteria.

Table 3.3: Logical Inconsistency Detection Results per Model

Model	DC (exp. 2)	MV (exp. 3)	TOV (exp. 2)	FP (exp. 0)	Total	Pass
Gemini	2	3	1 ×	0	6	Partial
GLM	2	3	2	0	7	Full
GPT-4o	2	3	1 ×	0	6	Partial
LLaMA	2	2 ×	1 ×	0	5	Partial
Mistral	2	3	2	0	7	Full

DC = Direct Conflict, MV = Monotonicity Violation, TOV = Tier Order Violation, FP = False Positives

Overall, GLM and Mistral demonstrated the strongest logical reasoning capability for this task, while LLaMA showed the most significant gaps. All models successfully avoided false positives, indicating that the prompt’s instruction to only flag genuine internal contradictions was well understood across the board. The primary differentiator between models was the completeness of pairwise comparison within the Tier Order and Monotonicity groups, where weaker models tended to stop after identifying the most salient violation rather than exhaustively checking all pairs.

3.5 Fetch Evidence

In this stage, the canonicalized triples are taken and for each triple the system retrieves related triples from the database, along with the general domain constraints that represent

the rules of the domain. These two outputs together serve as the evidence passed to the factual inconsistency detection stage.

For triples that received a LOW confidence score during canonicalization, no database lookup is performed, since their predicate could not be matched to any relationship type in the schema. However, these triples are not discarded; instead, a `predicate_known` flag is computed by checking whether the canonical predicate exists anywhere in the global set of relationship types in the database. This flag is passed forward to the detection stage, where it is used to distinguish an unverifiable triple, where the predicate itself is not defined in the schema at all.

For all other triples, the system executes 3 retrieval strategies against the database in a single round-trip query to minimise latency. The strategies are ordered by specificity: subject and predicate, subject only, object and predicate. The subject and predicate strategy is the strongest, as it constrains both the node and the relationship type simultaneously, and is therefore given the highest priority. The subject only and object and predicate strategies provide partial matches when one component of the triple is absent from the database. The system selects the result from the highest-priority strategy that returns at least one candidate, and records both the matched candidates and the strategy that produced them. Constraint nodes and Domain nodes are explicitly excluded from all four strategies, since they represent rules rather than facts and are fetched separately. String matching across all strategies is performed case-insensitively to avoid missing candidates.

The domain constraints are fetched once per pipeline run and cached globally to avoid repeated database calls across triples. They are retrieved by matching all nodes carrying the `Constraint` label and returning their properties, which encode the general rules of the domain that every triple must be checked against in the first pass of the factual inconsistency detection stage.

3.6 Detect Factual Inconsistency

This stage proceeds in two passes. The first pass compares each triple against the domain constraints fetched in the previous stage. For each triple, the matching constraints are first identified, and then all triples together with their constraints are given to an LLM, which is prompted to detect factual inconsistency rule violations. In the second pass, first the triples with low confidence is automatically labeled as *Unverifiable* where no db candidate is supporting this predicate. Second if the triple it's predicate is known and the search strategy is object+predicate then this is *fabricated fact* as the subject is not supported by db and it is fabricated. If triples has predicate known true and search strategy is subject only or null then this triple is sent to LLM without db evidence and LLM decide are they *correct* if the object is saying semantically that the answer is not known or out of context and this is correct, or *predicate mismatch* for triples with search strategy subject only or *fabricated fact* for triples with search strategy null. Triples with

search strategy subject+predicate is compared against their evidence triples retrieved from the database, again by having the LLM perform the comparison. The LLM assigns one of several verdict types: *rule violation* if the triple contradicts a domain constraint or an evidence triple fetched from the database; and *correct* when the entire triple is supported by the database.

This stage was tested in isolation by directly providing the detection function with 25 pre-labeled triples along with their evidence, bypassing the earlier pipeline stages entirely. The 25 triples were selected because they are all factually inconsistent and together cover every verdict type that the detection stage is designed to assign, ensuring that the evaluation exercises the full classification space rather than a random sample. The expected labels were: 12 rule violations from the constraint pass, 7 fabricated facts, 4 unverifiable triples, and 3 predicate mismatches. We evaluated five models — GLM-4.7, LLAMA3.3:70B, GPT-4.1, Gemini-2.5-Flash, and Mistral — and the results are reported in Table 3.4.

Table 3.4: Results of testing the factual inconsistency detection stage in isolation. The test set consisted of 25 pre-labeled triples covering all factually inconsistent verdict types.

Model	Errors	Description
GPT-4.1	0	All 25 triples correctly classified.
Gemini-2.5-Flash	0	All 25 triples correctly classified.
GLM-4.7	0	All 25 triples correctly classified.
LLAMA3.3:70B	1	One triple labeled as rule violation was incorrectly classified as fabricated fact.
Mistral	3	Two triples labeled as predicate mismatch were incorrectly classified as rule violations, and one triple labeled as fabricated fact was incorrectly classified as unverifiable.

Chapter 4

Future Work

Text

Appendix

Appendix A

Lists

AC	Acronym Without Citation
AC2	Acronym With Citation ?

List of Figures

3.1	Overview of the hallucination detection pipeline. The system processes the user query through five stages: triple extraction, canonicalization, logical inconsistency detection, evidence retrieval, and combined verification (symbolic constraint check followed by factual detection).	16
-----	---	----

Bibliography

- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), 1–56. <https://doi.org/10.1145/3703155>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 1–38. <https://doi.org/10.1145/3571730>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- Manakul, P., Liusie, A., & Gales, M. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 9004–9017). Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.557/>
- Lu, X., Welleck, S., Hessel, J., Jiang, L., Qin, L., West, P., Ammanabrolu, P., & Choi, Y. (2023). Adaptable logical control for large language models. *arXiv preprint arXiv:2306.00092*. <https://arxiv.org/abs/2306.00092>
- Sawczyn, A., Binkowski, J., Janiak, D., Gabrys, B., & Kajdanowicz, T. (2025). Fact-SelfCheck: Fact-level black-box hallucination detection for LLMs. *arXiv preprint arXiv:2501.07872*. <https://arxiv.org/abs/2501.07872>
- Yang, K., & Klein, D. (2021). FUDGE: Controlled text generation with future discriminators. In K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, & Y. Zhou (Eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 3511–3535). Association for Computational Linguistics. <https://aclanthology.org/2021.naacl-main.276/>

- Kale, S., & Alfeo, A. L. (2025). Lie to me: Knowledge graphs for robust hallucination self-detection in LLMs. *arXiv preprint arXiv:2509.12345*. <https://arxiv.org/abs/2509.12345>
- Wang, D., Peng, B., Liu, Z., Zhang, N., & Chen, J. (2022). TRACE back from the future: A probabilistic reasoning approach to controllable language generation. *arXiv preprint arXiv:2210.06647*. <https://arxiv.org/abs/2210.06647>
- Zhang, H., Song, H., Li, S., Zhou, M., & Song, D. (2023). Tractable control for autoregressive language generation. *arXiv preprint arXiv:2304.07438*. <https://arxiv.org/abs/2304.07438>
- Karatopak, E., Cekinel, R. F., & Yuret, D. (2024). Zero-knowledge LLM hallucination detection and mitigation through fine-grained cross-model consistency. *arXiv preprint arXiv:2405.16853*. <https://arxiv.org/abs/2405.16853>
- Fang, X., Huang, Z., Tian, Z., Fang, M., Pan, Z., Fang, Q., Wen, Z., Pan, H., & Li, D. (2024). Zero-resource hallucination detection for text generation via graph-based contextual knowledge triples modeling. *arXiv preprint arXiv:2409.11283*. <https://arxiv.org/abs/2409.11283>
- Lu, X., Welleck, S., Hessel, J., Jiang, L., Qin, L., West, P., Ammanabrolu, P., & Choi, Y. (2024). Sketch-guided constrained decoding for boosting blackbox large language models without logit access. *arXiv preprint arXiv:2401.09967*. <https://arxiv.org/abs/2401.09967>
- Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 12076–12100). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.emnlp-main.741>
- Hu, X., Ru, D., Qiu, L., Guo, Q., Zhang, T., Xu, Y., Luo, Y., Liu, P., Zhang, Y., & Zhang, Z. (2024). Knowledge-Centric Hallucination Detection. In Y. Al-Onaizan, M. Bansal, & Y. Chen (Eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 6953–6975). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-main.395>
- Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *CoRR*, abs/2305.11747. <https://doi.org/10.48550/arXiv.2305.11747>
- Dong, Z., Tang, T., Li, J., Zhao, W. X., & Wen, J.-R. (2023). BAMBOO: A Comprehensive Benchmark for Evaluating Long Text Modeling Capacities of Large Language Models. *arXiv preprint*, abs/2309.13345. <https://arxiv.org/abs/2309.13345>

- Lattimer, B. M., Chen, P., Zhang, X., & Yang, Y. (2023). Fast and Accurate Factual Inconsistency Detection Over Long Documents. *arXiv preprint*, abs/2310.13189. <https://arxiv.org/abs/2310.13189>
- Chen, S., Zhao, Y., Zhang, J., Chern, I.-C., Gao, S., Liu, P., & He, J. (2023). FELM: Benchmarking Factuality Evaluation of Large Language Models. *arXiv preprint*, abs/2310.00741. <https://arxiv.org/abs/2310.00741>
- Yang, S., Sun, R., & Wan, X. (2023). A New Benchmark and Reverse Validation Method for Passage-level Hallucination Detection. *arXiv preprint*, abs/2310.06498. <https://arxiv.org/abs/2310.06498>
- Feng, H., Fan, Y., Liu, X., Lin, T.-E., Yao, Z., Wu, Y., Huang, F., Li, Y., & Ma, Q. (2023). Improving Factual Consistency of Text Summarization by Adversarially Decoupling Comprehension and Embellishment Abilities of LLMs. *CoRR*, abs/2310.19347. <https://doi.org/10.48550/arXiv.2310.19347>
- Pan, L., Albalak, A., Wang, X., & Wang, W. Y. (2023). Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *arXiv preprint*, abs/2305.12295. <https://arxiv.org/abs/2305.12295>
- Sansford, H., Richardson, N., Petric Maretic, H., & Nait Saada, J. (2024). GraphEval: A Knowledge-Graph Based LLM Hallucination Evaluation Framework. In *Proceedings of the Workshop on Knowledge-infused Learning (KiL) at the 30th ACM KDD Conference*, Barcelona, Spain. *arXiv preprint* arXiv:2407.10793. <https://doi.org/10.48550/arXiv.2407.10793>
- Yu, S., Lee, C., Lee, H., Yoon, S. (2024). Controlled Text Generation for Black-box Language Models via Score-based Progressive Editor. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14215–14237, Bangkok, Thailand. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.acl-long.767>
- Chern, I.-C., Chern, S., Chen, S., Yuan, W., Feng, K., Zhou, C., He, J., Neubig, G., & Liu, P. (2023). FacTool: Factuality Detection in Generative AI - A Tool Augmented Framework for Multi-Task and Multi-Domain Scenarios. *arXiv preprint*, abs/2307.13528. <https://arxiv.org/abs/2307.13528>
- Zhang, B., & Soh, H. (2024). Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9820–9836, Miami, Florida, USA. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.548/>